

Day 38 — Line-by-Line Syntax Explanation

day38_regularization.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and a real (rerun, not fabricated) graph showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 6, Day 3 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (BCE loss, L2 weight decay, inverted dropout, early stopping's patience rule), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the extended training work cycle, and the script's own real output graph.

MATH box legend: $\times$ = matrix multiply / times $W1, b1$ = weights/bias of layer 1, etc. X^T = transpose of X
 $\text{mean}[...] =$ average over all examples m = number of training examples lam = L2 strength keep_prob = dropout survival probability

Segment 0 — Imports and Global Setup (lines 31–37)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

np.random.seed(42)
```

Identical setup pattern to Days 36-37: **np** for all array math, **matplotlib.use("Agg")** before importing **pyplot** so charts render straight to PNG files instead of a GUI window, **make_moons** imported directly to build this lesson's deliberately small dataset, and **np.random.seed(42)** fixing every subsequent "random" NumPy call so this script's numbers are exactly reproducible.

Segment 1 — Shared Helpers and the Overfit Dataset (lines 44–63)

MATH

$$\text{sigmoid}(z) = 1 / (1 + e^{-z})$$

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

Character-for-character identical to Day 36/37's **sigmoid** — the activation function used at both layers of every network in this lesson, unchanged.

MATH

$$\text{BCE}(\text{pred}, \text{target}) = -\text{mean}[\text{target} \times \log(\text{pred}) + (1-\text{target}) \times \log(1-\text{pred})]$$

```
def bce_loss(pred, target, eps=1e-9):  
    pred = np.clip(pred, eps, 1 - eps)  
    return -np.mean(target * np.log(pred) + (1 - target) * np.log(1 - pred))
```

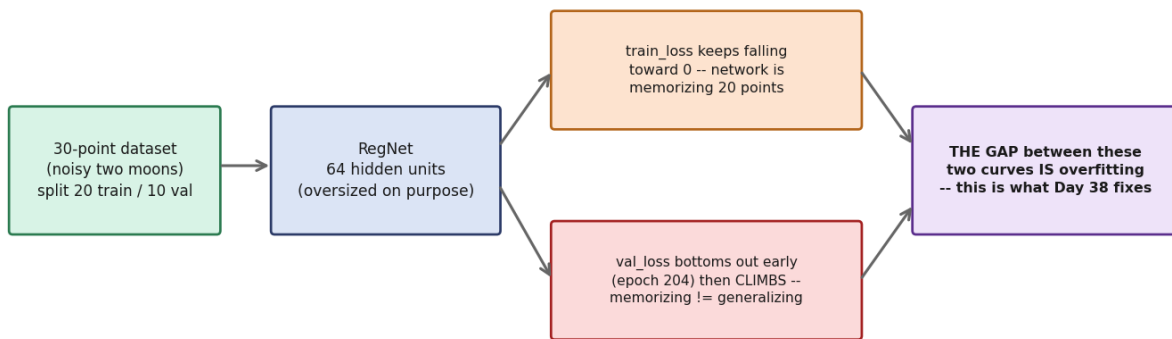
The same binary cross-entropy function from Day 37, reused as-is: **np.clip(...)** keeps every prediction inside [eps, 1-eps] so **np.log(0)** never happens, and the return line is the BCE formula in one vectorized expression.

```
def make_overfit_dataset():  
    X, y = make_moons(n_samples=30, noise=0.35, random_state=42)  
    y = y.reshape(-1, 1)  
    X_train, y_train = X[:20], y[:20]  
    X_val, y_val = X[20:], y[20:]  
    return X_train, y_train, X_val, y_val
```

make_moons(n_samples=30, noise=0.35, ...) generates only 30 points total with heavy label noise — deliberately small and messy. **y.reshape(-1, 1)** turns the 1D label array into a column vector (shape (30, 1)) so it lines up with the network's (m, 1) output shape. **X[:20]** and **X[20:]** are plain array slicing — the first 20 rows become the training set, the remaining 10 become a validation set the network never trains on. This exact split is what makes overfitting detectable at all: nothing here trains the network yet, it only carves up the data.

SYSTEM DIAGRAM

The overfitting problem, made visible on purpose (Segment 3-4)



The dataset and network from this segment, and the exact gap that defines overfitting: train loss keeps falling while val loss, measured on the 10 points this segment set aside, bottoms out and climbs back up.

Segment 2 — RegNet.__init__ (lines 70–83)

```
class RegNet:
    """Same 2-layer sigmoid network as Day 36/37's TinyNet/Net, extended
    with two optional regularizers: L2 weight decay (lam) and inverted
    dropout (keep_prob on the hidden layer)."""

    def __init__(self, n_in, n_hidden, lr=0.5,
                  lam=0.0, keep_prob=1.0, seed=42):
        rng = np.random.RandomState(seed)
        self.W1 = rng.randn(n_in, n_hidden) * 0.5
        self.b1 = np.zeros(n_hidden)
        self.W2 = rng.randn(n_hidden, 1) * 0.5
        self.b2 = np.zeros(1)
```

The same weight-initialization pattern as Day 37's **Net**: an independent **RandomState(seed)** so every configuration in this lesson (no regularization, L2, dropout, early stopping) starts from *identical* weights — the only fair way to compare them. **lam=0.0** and **keep_prob=1.0** are the DEFAULT arguments: with no arguments overridden, this class behaves exactly like Day 37's plain **Net**, with neither regularizer active.

```
        self.lr = lr
        self.lam = lam
        self.keep_prob = keep_prob
```

Three hyperparameters stored directly on the instance: **lr** (learning rate, unchanged from Day 37), **lam** (L2 strength — 0 means "off"), and **keep_prob** (dropout survival probability — 1.0 means "off," since no unit ever gets dropped at probability 1). One class, one constructor, four completely different lesson configurations just by varying these two numbers.

Segment 3 — RegNet.forward (lines 85–95)

MATH

$$z1 = X \times W1 + b1 \quad h1 = \text{sigmoid}(z1)$$

```
def forward(self, X, training=False):
    self.z1 = X @ self.W1 + self.b1
    self.h1 = sigmoid(self.z1)
```

Identical to every forward pass so far through the first layer: **self.h1** is the hidden layer's activation BEFORE any dropout is applied — kept as a separate variable from what actually continues to layer 2, which is the new piece this segment introduces.

MATH

$$\text{mask} = (\text{rand}(\text{shape}) < \text{keep_prob}) / \text{keep_prob} \quad (\text{only during training})$$
$$a1 = h1 \times \text{mask} \quad (a1 = h1 \text{ when not training})$$

```
if training and self.keep_prob < 1.0:
    keep = self.keep_prob
    self.mask = (np.random.rand(*self.h1.shape) < keep) / keep
    self.a1 = self.h1 * self.mask
else:
    self.mask = None
    self.a1 = self.h1
```

The **training** flag decides everything here: dropout only ever activates when BOTH **training=True** AND **keep_prob** is below 1.0. **np.random.rand(*self.h1.shape)** draws one random number per hidden unit, uniformly between 0 and 1; comparing **< keep_prob** turns that into a boolean array (True = "keep this unit"), and dividing by **keep_prob** in the SAME expression rescales survivors up — this single line is the entire "inverted dropout" trick. **self.a1 = self.h1 * self.mask** zeroes out the dropped units elementwise. At evaluation time (**training=False**), the **else** branch runs instead: **self.mask = None** and **self.a1** is just **self.h1** unchanged — the full network, no units missing.

MATH

$$z2 = a1 \times W2 + b2 \quad a2 = \text{sigmoid}(z2) = y_{\text{pred}}$$

```
self.z2 = self.a1 @ self.W2 + self.b2
self.a2 = sigmoid(self.z2)
return self.a2
```

The rest of the forward pass is unchanged from Day 36/37 — it simply uses **self.a1**, whatever that happens to be (masked or not), without needing to know or care whether dropout ran.

Segment 4 — RegNet.backward (lines 97–115)

MATH

$$\begin{aligned} dz2 &= a2 - y & dW2 &= (1/m) \times a1^T \times dz2 & db2 &= \text{mean}(dz2) \\ da1 &= dz2 \times W2^T \end{aligned}$$

```
def backward(self, X, y_col):
    m = X.shape[0]
    dz2 = self.a2 - y_col
    dW2 = self.a1.T @ dz2 / m
    db2 = dz2.mean(axis=0)
    da1 = dz2 @ self.W2.T
```

Identical to Day 36/37's backward pass so far, through the output layer's gradients — note **dW2** uses **self.a1** (the possibly MASKED activation), automatically wiring dropout's effect into this gradient without any extra code here.

MATH

$$\begin{aligned} dh1 &= da1 \times \text{mask} \quad (\text{same mask as forward()}; dh1 = da1 \text{ if no dropout}) \\ dz1 &= dh1 \times h1 \times (1-h1) \quad (\text{sigmoid derivative, as in every prior lesson}) \\ dW1 &= (1/m) \times X^T \times dz1 & db1 &= \text{mean}(dz1) \end{aligned}$$

```
dh1 = da1 * self.mask if self.mask is not None else da1
dz1 = dh1 * self.h1 * (1 - self.h1)
dW1 = X.T @ dz1 / m
db1 = dz1.mean(axis=0)
```

The one new line dropout requires on the way back: a conditional expression (**value_if_true if condition else value_if_false**, all on one line) that multiplies **da1** by the EXACT SAME **self.mask** saved during the forward pass, if one exists. This is essential — a hidden unit that was zeroed out contributed nothing to the loss, so it must also receive zero gradient here; reusing the identical mask (rather than sampling a new one) is what guarantees that. When there's no mask (**training=False** or no dropout configured), **dh1** is just **da1**, unchanged.

MATH

$$\begin{aligned} \text{penalty} &= (lam/2) \times \text{sum}(W^2) & d(\text{penalty})/dW &= lam \times W \\ dW1 &= dW1 + lam \times W1 & dW2 &= dW2 + lam \times W2 \end{aligned}$$

```
# L2 weight decay: add lam * W directly to the weight gradients only
# (biases are conventionally left unregularized).
dW1 = dW1 + self.lam * self.W1
dW2 = dW2 + self.lam * self.W2
```

L2's entire implementation, in two lines: the current weight matrix, scaled by **lam**, is added directly to that weight's own gradient — the derivative of the penalty term **(lam/2) × sum(W²)** with respect to **W** is exactly **lam × W**. Only **dW1/dW2** are touched; **db1/db2** (the bias gradients) are never modified — biases are conventionally

excluded from L2 by this codebase's own comment.

MATH

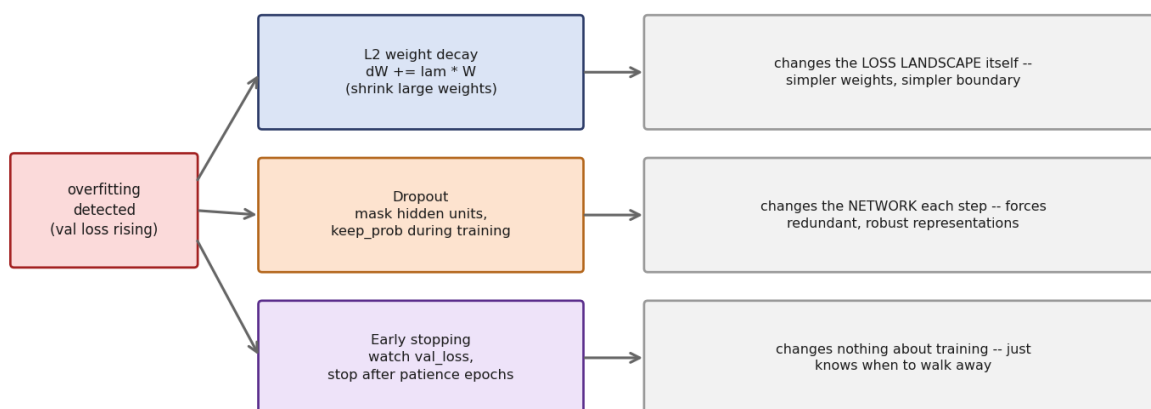
$param \leftarrow lr \times gradient$ (for each of $W1, b1, W2, b2$)

```
self.W1 -= self.lr * dw1
self.b1 -= self.lr * db1
self.W2 -= self.lr * dw2
self.b2 -= self.lr * db2
```

The same plain gradient-descent update from Day 36's **TinyNet** — no optimizer pluggability this time (Day 37's Momentum/Adam machinery isn't reused here, to keep this lesson's comparison isolated to regularization alone, not optimizer choice).

SYSTEM DIAGRAM

Three regularizers, three different mechanisms (Segments 5-7)



The two regularizers built so far, plus early stopping (Segment 5): three different intervention points in the exact same forward/backward pass just read line by line.

Segment 5 — RegNet.train (lines 117–144)

```
def train(self, X_train, y_train, X_val, y_val, epochs, patience=None):
    train_losses, val_losses = [], []
    best_val = np.inf
    best_epoch = 0
    epochs_no_improve = 0
    stopped_at = epochs
```

Five tracking variables set up before the loop starts. **train_losses, val_losses = [], []** unpacks two empty lists in one line. **best_val = np.inf** starts the "best so far" value at infinity so that literally any real loss value will count as an improvement the first time it's checked. **stopped_at = epochs** defaults to the FULL epoch count, only overwritten if patience actually triggers early — meaning this same variable correctly reports "ran to completion" when patience never runs out.

```
for epoch in range(epochs):
    pred_train = self.forward(X_train, training=True)
    self.backward(X_train, y_train)
```

training=True here is what switches dropout on for this step (Segment 3) — note the forward pass's return value, **pred_train**, is discarded and never used; only its SIDE EFFECT (setting **self.a1**, **self.a2**, etc. for **backward()** to read) matters at this point in the loop.

```
train_pred_eval = self.forward(X_train, training=False)
val_pred = self.forward(X_val, training=False)
train_loss = bce_loss(train_pred_eval, y_train)
val_loss = bce_loss(val_pred, y_val)
train_losses.append(train_loss)
val_losses.append(val_loss)
```

A SECOND, separate forward pass on the training data — this time with **training=False** — exists purely to measure the loss without dropout's randomness distorting the number being tracked; the update already happened in the previous block using the training-mode forward pass. **val_pred**'s forward pass also uses **training=False**, since the validation set is never used to compute gradients at all, only to measure generalization.

MATH

if val_loss < best_val - 0.0001: reset patience counter, remember this epoch
else: patience counter += 1

```
if val_loss < best_val - 1e-4:
    best_val = val_loss
    best_epoch = epoch
    epochs_no_improve = 0
else:
    epochs_no_improve += 1
```

The patience mechanism's core check. **val_loss < best_val - 1e-4** requires the new loss to beat the best-so-far by more than a tiny margin (**1e-4**) — this guards against resetting the patience counter on statistically meaningless micro-noise. If it's a real improvement, **best_val/best_epoch** update and the counter resets to 0; otherwise the counter increments by one.

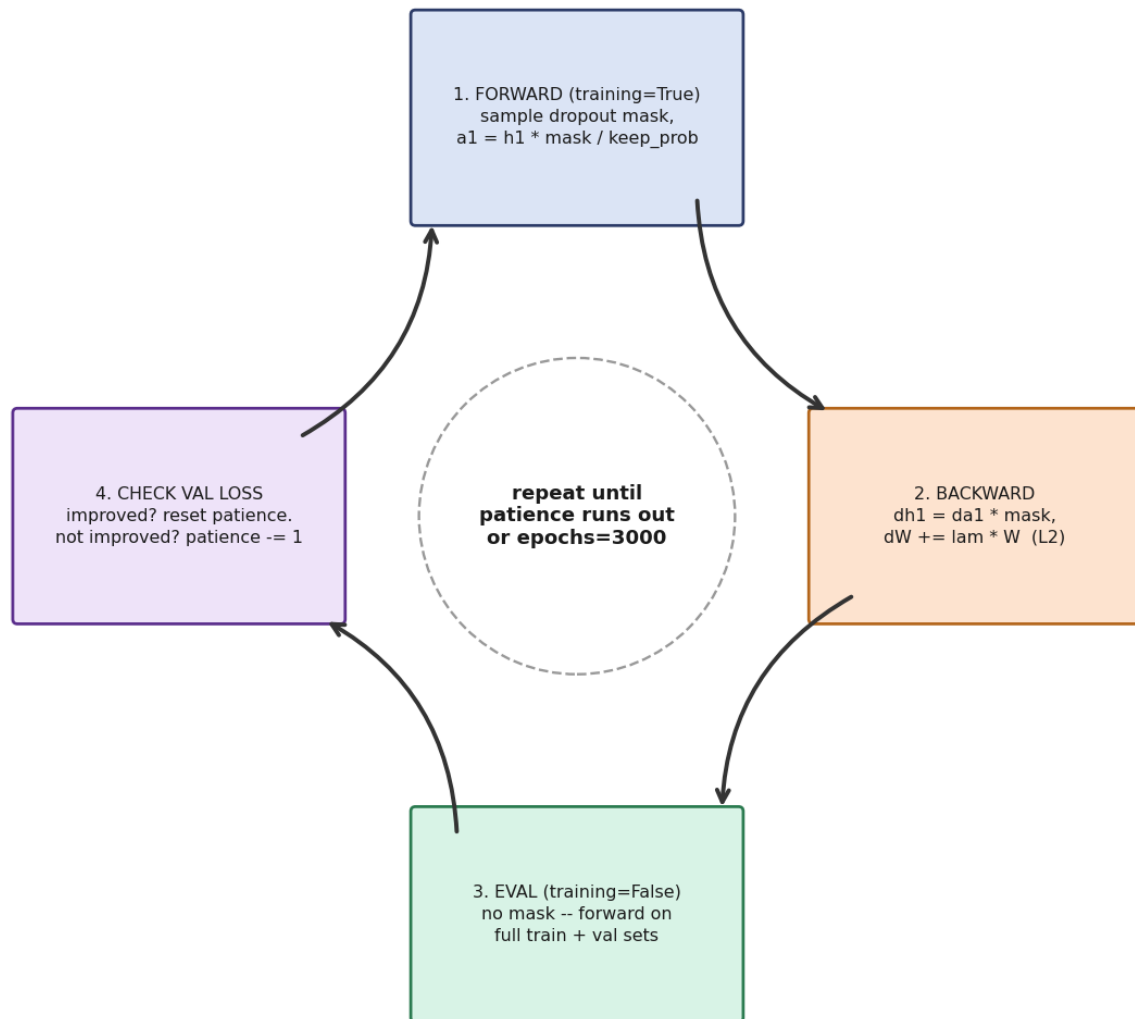
```
        if patience is not None and epochs_no_improve >= patience:
            stopped_at = epoch + 1
            break

    return train_losses, val_losses, best_epoch, stopped_at
```

patience is not None lets this same **train()** method serve BOTH the early-stopping demo (**patience=200**) and every other demo (**patience=None**, meaning this check is always False and training runs the full **epochs** count) — one method, no duplicated loop. **break** exits the **for** loop immediately once patience is exhausted; everything already appended to **train_losses/val_losses** up to that point is returned as-is.

SYSTEM DIAGRAM

The training work cycle, extended for regularization (train())



This segment's loop as a work cycle — the same 4-stage shape as Day 37's, extended with the dropout mask (stage 1), L2's gradient penalty (stage 2), a training-mode-off evaluation pass (stage 3), and the new patience check (stage 4) that can end the whole cycle early.

Segment 6 — RegNet.accuracy (lines 146–148)

MATH

$$accuracy = mean([(y_pred \geq 0.5) == y])$$

```
def accuracy(self, X, y):  
    pred = (self.forward(X, training=False) >= 0.5).astype(int)  
    return (pred == y).mean()
```

The same accuracy-via-boolean-mean trick from Days 36-37: **training=False** ensures accuracy is always measured on the full, non-dropped network regardless of which configuration is being evaluated; threshold at 0.5, cast True/False to 1/0, compare against true labels, average the matches.

Segment 7 — The Four Demos (lines 155–260)

```
def demo_overfitting():
    X_train, y_train, X_val, y_val = make_overfit_dataset()
    net = RegNet(n_in=2, n_hidden=64, lr=0.5, lam=0.0, keep_prob=1.0)
    train_losses, val_losses, best_epoch, _ = net.train(
        X_train, y_train, X_val, y_val, epochs=3000
    )
```

The baseline: **lam=0.0, keep_prob=1.0** means neither regularizer is active — this call is functionally identical to Day 37's plain **Net**, deliberately trained for a long budget (3000 epochs) on only 20 points with 64 hidden units, so overfitting has every chance to appear. The underscore **_** in the unpacking discards **stopped_at**, since **patience** wasn't passed (defaults to **None**) and it isn't needed here.

```
def demo_l2():
    ...
    net = RegNet(n_in=2, n_hidden=64, lr=0.5, lam=0.02, keep_prob=1.0)
    train_losses, val_losses, best_epoch, _ = net.train(
        X_train, y_train, X_val, y_val, epochs=3000
    )
```

Identical network size, identical dataset, identical epoch budget to **demo_overfitting()** — the ONLY thing that changed is **lam=0.02**. This is the entire point of structuring the demos this way: any difference in the resulting curves is attributable to L2 alone, nothing else.

```
def demo_dropout():
    ...
    net = RegNet(n_in=2, n_hidden=64, lr=0.5, lam=0.0, keep_prob=0.6)
    train_losses, val_losses, best_epoch, _ = net.train(
        X_train, y_train, X_val, y_val, epochs=3000
    )
```

Same pattern again: only **keep_prob=0.6** changed from the baseline (40% of hidden units dropped every training step). **lam** is explicitly back to 0.0 — this run isolates dropout's effect alone, with no L2 mixed in.

```
def demo_early_stopping():
    ...
    net = RegNet(n_in=2, n_hidden=64, lr=0.5, lam=0.0, keep_prob=1.0)
    train_losses, val_losses, best_epoch, stopped_at = net.train(
        X_train, y_train, X_val, y_val, epochs=3000, patience=200
    )
```

Back to the exact same unregularized network as **demo_overfitting()** (**lam=0.0, keep_prob=1.0**) — the ONLY difference this time is **patience=200** passed to **train()**. This time **stopped_at** IS captured (not discarded), since this demo's whole point is reporting when training actually stopped.

```
print(f"Epoch 0:   train_loss={train_losses[0]:.4f}  val_loss={val_losses[0]:.4f}")
print(f"Epoch {best_epoch}: val_loss hits its BEST value = {val_losses[best_epoch]:.4f}")
print(f"Final train accuracy: {net.accuracy(X_train, y_train):.3f}")
```

Every demo ends by printing a handful of specific epochs and final accuracies using f-string indexing — **train_losses[0]** is the very first epoch's loss, **val_losses[best_epoch]** uses the **best_epoch** value **train()** returned to look up that exact epoch's val_loss directly, rather than recomputing anything.

Segment 8 — demo_comparison and note_on_pytorch (lines 262–299)

```
def demo_comparison(results):
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    titles = ["No regularization", "L2 weight decay (lam=0.02)",
              "Dropout (keep_prob=0.6)", "Early stopping (patience=200)"]
    zipped = zip(axes.flat, titles, results)
    for ax, title, (train_losses, val_losses) in zipped:
```

plt.subplots(2, 2, ...) creates a 2x2 grid of subplot axes in one call. **axes.flat** flattens that 2x2 grid into a simple 1D sequence of 4 axes so it can be walked with one loop instead of nested loops. **zip(axes.flat, titles, results)** walks THREE sequences in lockstep — one axis, one title, one (train, val) loss pair per iteration — and the nested **(train_losses, val_losses)** immediately unpacks each result tuple as it's pulled from **results**.

```
    ax.plot(train_losses, label="train", color="#378ADD", linewidth=1.4)
    ax.plot(val_losses, label="val", color="#D85A30", linewidth=1.4)
    ax.set_title(title, fontsize=10)
    ax.legend(fontsize=8)
    ax.set_ylim(0, 1.4)
```

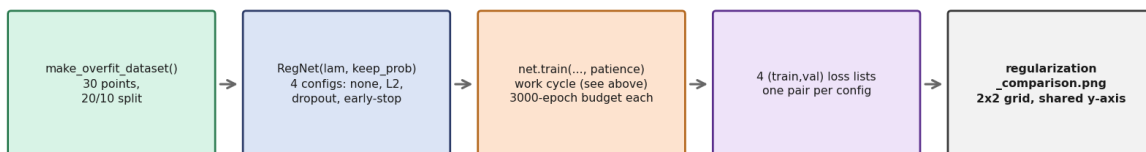
Two lines per subplot, one per loss list, with a shared, fixed y-axis range (**set_ylim(0, 1.4)**) across all four panels — essential for an honest visual comparison, since without a shared scale, a flatter-LOOKING curve could just be an auto-scaled axis, not a genuinely smaller loss.

```
plt.savefig("regularization_comparison.png", dpi=110)
```

Saves the completed 2x2 figure straight to a PNG file — this exact call is what produced the real graph on the next page; nothing about that image was hand-drawn or edited afterward.

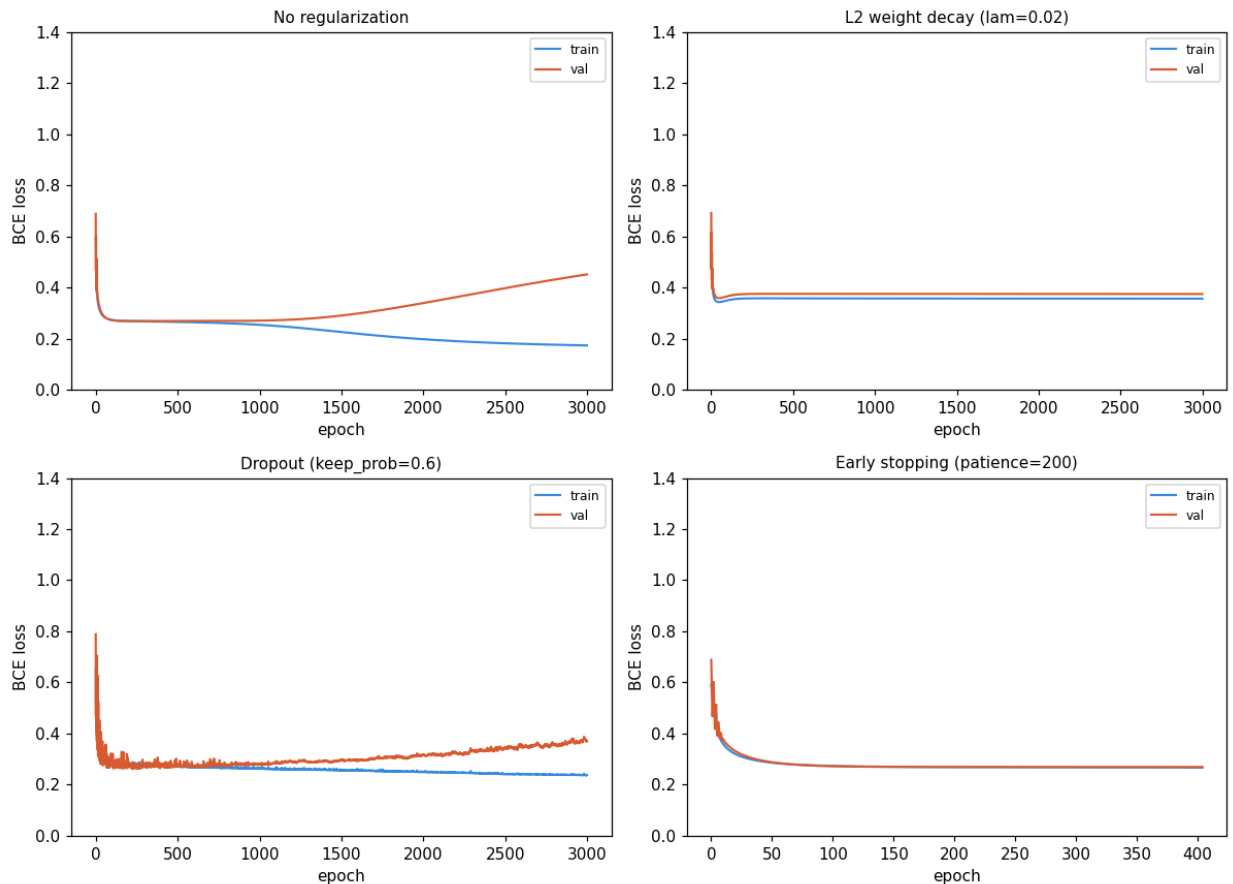
SYSTEM DIAGRAM

Segment 8 pipeline: how code + math becomes the final comparison chart



Zooming out: the full pipeline this segment runs end to end, from the 20/10 split through four independent training work cycles to the final comparison chart — every box here is code you've now read line by line.

REAL GRAPH (script's own output)



Not a mockup — this is regularization_comparison.png, this exact script rerun fresh. Top-left panel: val loss (orange) climbing back up after epoch ~200 while train loss (blue) keeps falling — the overfitting this whole lesson exists to fix. The other three panels are the same axes, same y-limit, three different fixes.

```
nn.Dropout(p=0.4)
# keep_prob=0.6 here

optim.SGD(params, lr=0.5, weight_decay=0.02)
# L2, same as lam=0.02 here
```

note_on_pytorch() is just **print()** calls containing plain text — not executable code, and PyTorch is never imported anywhere in this file. These are reference lines mapping each hand-written mechanism to its PyTorch equivalent, with an explicit reminder in the comment that PyTorch's dropout argument uses the opposite convention (drop probability, not keep probability) from this script's **keep_prob**.

Segment 9 — main() and the Entry-Point Guard (lines 301–317)

```
def main():
    train_losses_0, val_losses_0, best_epoch_0 = demo_overfitting()
    train_losses_l2, val_losses_l2, best_epoch_l2 = demo_l2()
    train_losses_do, val_losses_do, best_epoch_do = demo_dropout()
    train_losses_es, val_losses_es, best_epoch_es, stopped_at = demo_early_stopping()
    demo_comparison([
        (train_losses_0, val_losses_0),
        (train_losses_l2, val_losses_l2),
        (train_losses_do, val_losses_do),
        (train_losses_es, val_losses_es),
    ])
    note_on_pytorch()
```

Same pattern as Days 36-37: one function calling every demo in the exact order they should run, then explicitly re-packaging each demo's (train, val) loss pair into a list for **demo_comparison()** — the single source of truth for "what does running this script do," with the data flow from individual demos into the final chart made fully explicit right here.

```
if __name__ == "__main__":
    main()
```

The same entry-point guard as every script in this roadmap: `__name__` only equals "`__main__`" when this file is run directly, so **main()** won't silently re-execute if **RegNet** or **bce_loss** is ever imported from this file elsewhere.

Quick Syntax Glossary — New or Reinforced Constructs

`np.random.rand(*shape)` — Draws uniform random numbers in `[0, 1)` with the given shape — the `*` unpacks a shape tuple into separate arguments. Comparing the result with `<` against a probability turns it into a boolean mask.

`value_if_true if condition else value_if_false` — A conditional (ternary) expression: evaluates to one value or another, all on one line, without needing a full `if/else` block — used here to pick between the dropout mask and the plain gradient.

`a, b = [], []` — Unpacking assignment used to initialize two separate empty lists in one line.

`np.inf` — NumPy's representation of positive infinity — used as a starting 'best so far' value that any real number will beat on the first comparison.

`a, b = a, b (default arguments)` — Function parameters like **`patience=None`** let the same method serve two purposes (early-stopping runs and normal runs) depending on whether the caller supplies a value.

`axes.flat` — Flattens a 2D grid of matplotlib subplot axes (from `plt.subplots(2, 2, ...)`) into a simple 1D iterable, so a single for-loop can walk all 4 panels.

`zip(a, b, c)` — Walks three sequences in lockstep, yielding one tuple of `(a_i, b_i, c_i)` per iteration — used here to pair each subplot axis with its title and its (train, val) loss data simultaneously.

nested unpacking: `for x, (y, z) in zip(...)` — A tuple pulled from a zipped sequence can itself be immediately unpacked into two more variables in the same for-loop header.

`break` — Exits a for/while loop immediately, skipping any remaining iterations — used here to stop training the instant patience runs out, without an extra flag variable.

underscore `_` in unpacking — A conventional throwaway variable name for a return value that's deliberately not needed — used here to discard `train()`'s `stopped_at` when patience isn't being used.

End of Day 38 syntax reference. For the regularization concepts these lines implement, see the main Day 38 study guide PDF.